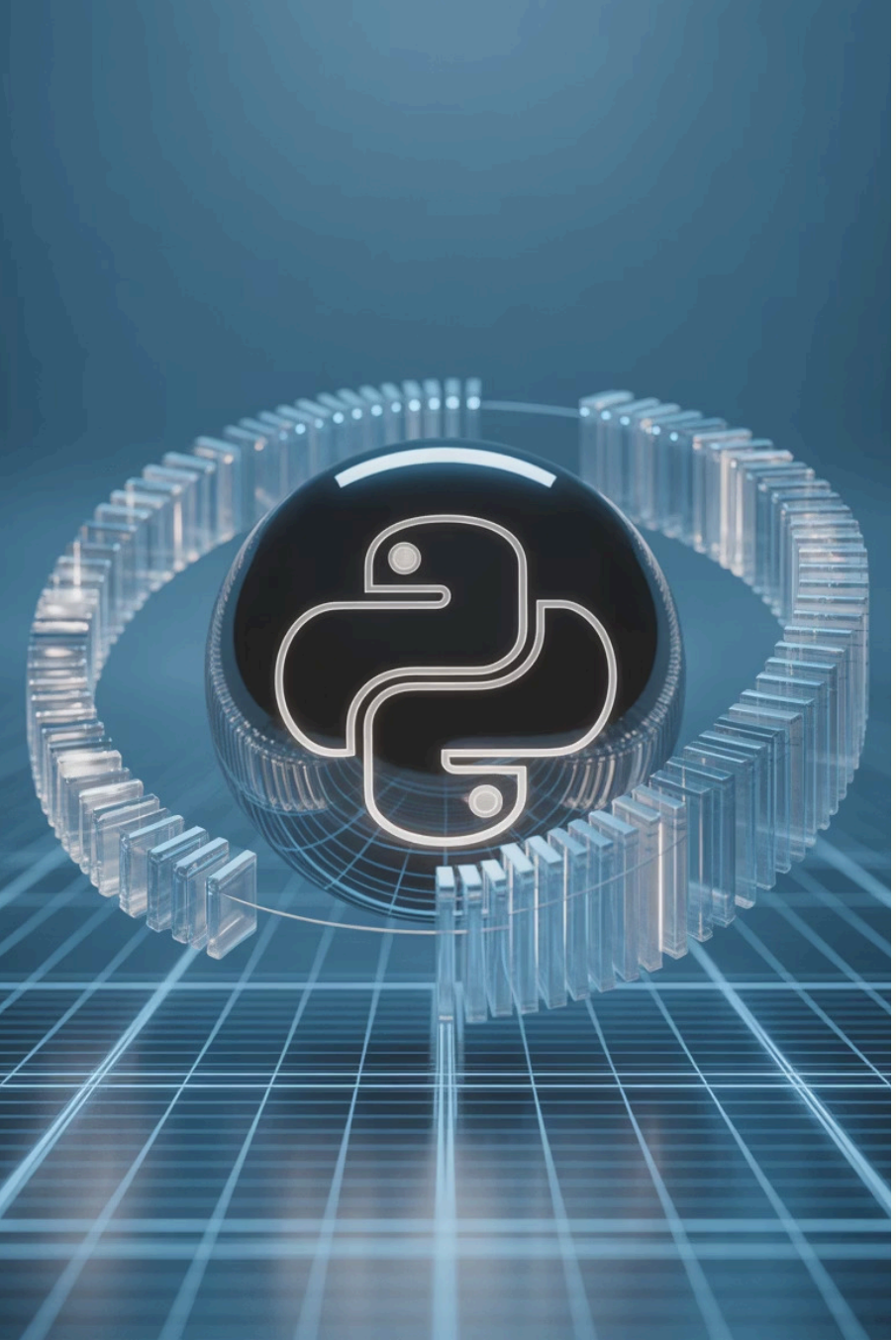


Tema 3: Tipos de Datos Avanzados en Python

2DAW - Valle del Miro - Curso 2025-2026

Agenda

1

Introducción a los tipos de datos avanzados

Visión general de los tipos de datos que estudiaremos y su importancia en la programación con Python.

2

Cadenas de texto (str)

Manipulación de texto: creación, formateo y métodos principales.

3

Listas (list)

Colecciones ordenadas y modificables para almacenar múltiples elementos.

4

Diccionarios (dict)

Estructuras de datos con pares clave-valor para búsquedas eficientes.

5

Tuplas (tuple)

Colecciones ordenadas e inmutables para datos que no deben cambiar.

6

Conjuntos (set y frozenset)

Colecciones sin duplicados para operaciones matemáticas de conjuntos.

Introducción a los Tipos de Datos Avanzados

En programación, los **datos** son la base de cualquier programa. Todo lo que hace un programa consiste en **recibir información**, **procesarla** y **mostrar un resultado**. Python ofrece diferentes **tipos de datos** para almacenar y trabajar con la información de la manera más adecuada.

Cadenas (str)

Para trabajar con texto.

Listas (list)

Colecciones ordenadas y modificables de datos.

Diccionarios (dict)

Estructuras que almacenan datos en pares clave–valor.

Tuplas (tuple)

Colecciones ordenadas pero inmutables.

Conjuntos (set)

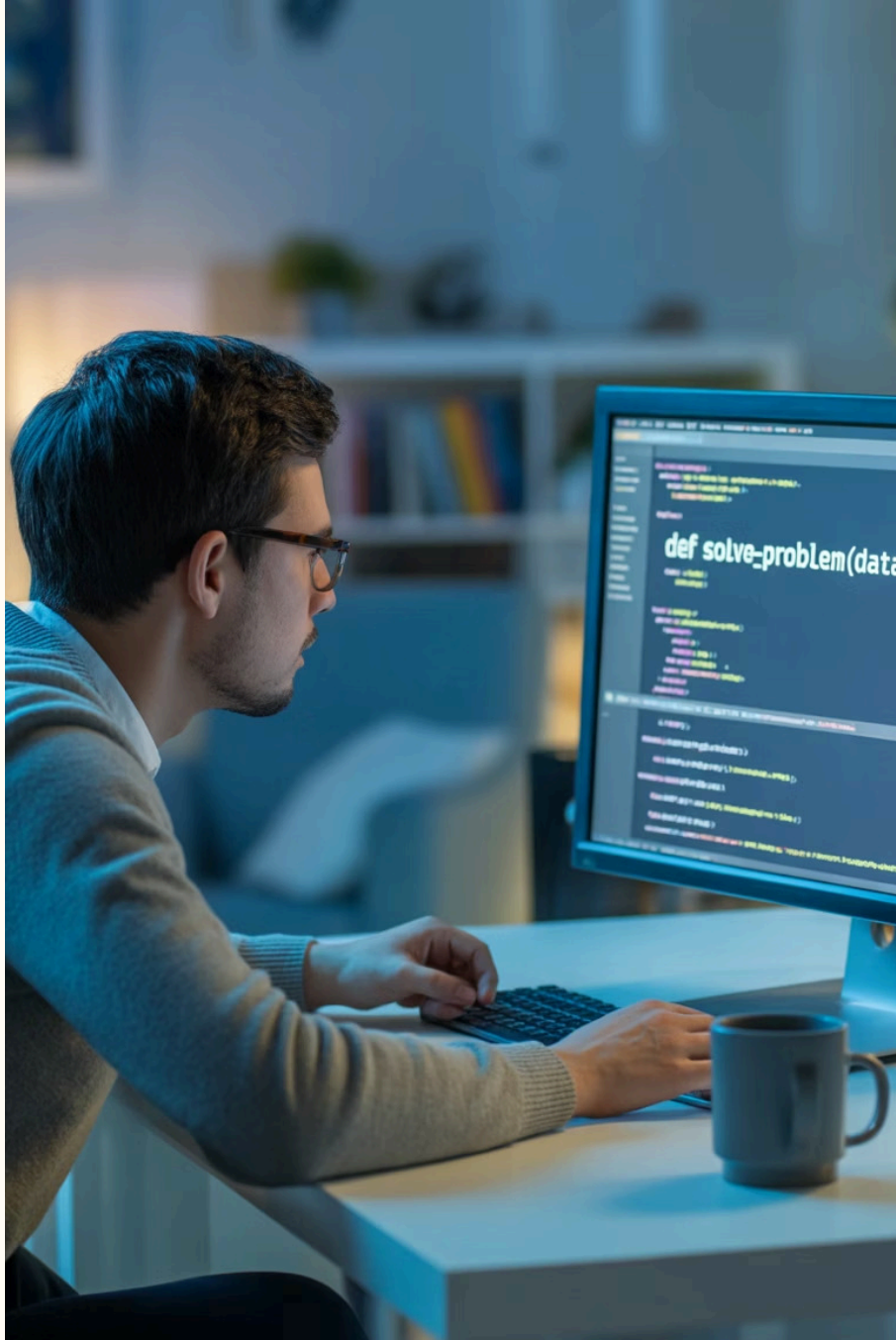
Colecciones sin duplicados, ideales para operaciones matemáticas.

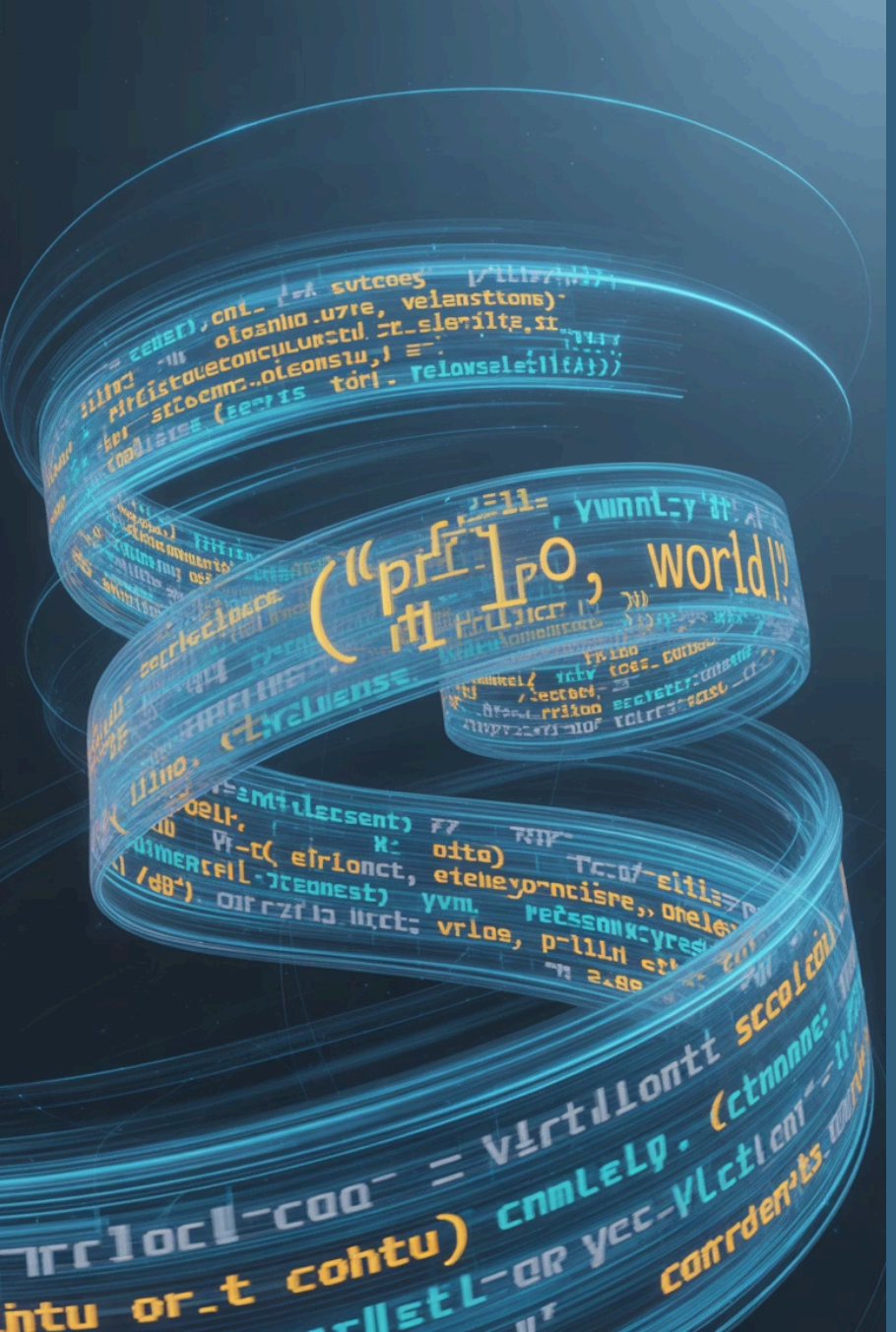
¿Por qué son importantes?

Aprender a usar correctamente estos tipos es fundamental porque son la **base de la mayoría de los programas en Python**. Gracias a ellos podremos:

- Procesar textos y archivos
- Guardar listas de elementos como notas de un examen o productos en un carrito de compra
- Contar la frecuencia de palabras en un texto
- Realizar búsquedas rápidas sin duplicados
- Representar estructuras complejas de datos como usuarios, redes, tablas, etc.

Al final de esta unidad, serás capaz de **elegir el tipo adecuado según la situación** y de **manipular la información eficientemente**.





Cadenas de texto (str)



¿Qué son las cadenas de texto?

El texto es uno de los datos más comunes en programación: nombres de personas, direcciones, contraseñas, correos electrónicos, mensajes en una red social, etc.

Definición

Una **cadena** es una secuencia de caracteres que representan texto. En Python, **TODO lo que se encuentre entre comillas** (simples o dobles) se considerará una cadena de texto.

Ejemplos: "Hola", "1234", "python@correo.com"

Características

- Cada carácter tiene una posición concreta (índice)
- Son **inmutables**: una vez creadas, no se pueden modificar en el sitio
- Si queremos un texto "cambiado", Python crea una nueva cadena

Cómo declarar cadenas

Con comillas simples o dobles

```
s1 = 'Hola mundo'
s2 = "Hola mundo"

frase = "Ella dijo: 'Hola a todos'"
frase = 'Ella dijo: "Hola a todos"'
```

Con comillas triples para texto multilínea

```
mensaje = """Hola,
este es un texto
que ocupa varias líneas."""
```

Convertir a cadena: str(objeto)

```
numero = 123
cadena_numero = str(numero)
print("El número es: " + cadena_numero)
```

Inmutabilidad de las cadenas

En Python, las cadenas (str) son **inmutables**, lo que significa que **no se pueden modificar una vez creadas**.

Cuando parece que "modificamos" una cadena, en realidad Python está **creando una nueva cadena en memoria** y, después, **la variable pasa a apuntar a esa nueva cadena**.

```
texto = "Hola"
texto = "Adiós"  # No modifica "Hola", crea "Adiós"
```

Si intentamos modificar directamente:

```
mensaje = "Hola"
mensaje[0] = "M"  # ¡ERROR!
```

Python mostrará: `TypeError: 'str' object does not support item assignment`





Índices y slicing en cadenas

Uso de índices

Como las cadenas son **secuencias ordenadas**, podemos acceder a cada carácter usando **índices**.

- El **primer carácter** tiene índice 0
- Los **índices negativos** cuentan desde el final (-1 es el último carácter)

```
s = "Python"
s[0]  # 'P'
s[-1]  # 'n'
```

¡		H	O	L	A		M	U	N	D	O		!
0	1	2	3	4	5	6	7	8	9	10	11	12	13
0	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Slicing: subcadenas

El **slicing** permite obtener partes de una cadena usando la sintaxis:

```
cadena[inicio:fin:paso]
```

Donde:

- **inicio** → índice donde empieza (incluido)
- **fin** → índice donde termina (**excluido**)
- **paso** → salto entre caracteres (opcional)

```
texto = "Python"
texto[0:4] # "Pyth"
texto[:4] # "Pyth"
texto[2:] # "thon"
texto[::-1] # "nohtyP" (invertido)
```

Operaciones comunes con cadenas

Operación	Ejemplo	Resultado
Concatenación	"Hola " + "Mundo"	"Hola Mundo"
Repetición	"ha" * 3	"hahaha"
Pertenencia	"py" in "python"	True
Longitud	len("hola")	4

Ejemplo práctico

```
saludo = "Hola"
nombre = "María"
mensaje = saludo + " " + nombre + "!"
print(mensaje) # "Hola María!"

risa = "ja" * 5
print(risa) # "jajajajaja"

print("Py" in "Python") # True
```

Formateo de cadenas: f-strings

Las f-strings (desde Python 3.6) son la forma más moderna y eficiente de crear cadenas con valores variables.

Forma tradicional (concatenación)

```
nombre = "Ana"
nota = 9.25
mensaje = "Alumno: " + nombre + " - Nota: " + str(nota)
print(mensaje)
```

Problemas:

- Necesitamos convertir a texto manualmente (str(nota))
- El código se vuelve largo y confuso
- Es fácil cometer errores con espacios y símbolos

Con f-strings

```
nombre = "Ana"
nota = 9.25
mensaje = f"Alumno: {nombre} - Nota: {nota}"
print(mensaje)
```

Ventajas:

- Más limpio y legible
- Conversión automática a texto
- Permite expresiones dentro de las llaves

```
a = 10
b = 3
print(f"La suma de {a} + {b} es {a + b}")
print(f"El doble de {a} es {a * 2}")
```

Regla de oro: Siempre que necesites construir cadenas con variables, **usa f-strings**. Te ahorrarán tiempo y harán tu código mucho más limpio.

Cadenas crudas en Python (Raw strings)

Las **cadenas crudas (raw strings)** son una forma de decirle a Python que trate los caracteres de barra invertida (`\`) como literales, en lugar de como caracteres de escape. Esto es muy útil para evitar problemas cuando las barras invertidas son parte del texto, como en rutas de archivos de Windows o expresiones regulares.

Una **cadena cruda** es una cadena en la que **Python no interpreta las secuencias de escape**, sino que toma el contenido **tal cual se escribe**. Esto se logra añadiendo **una letra** `r` **o** `R` **justo antes de las comillas** que definen la cadena.

```
# Sin cadena cruda, el carácter '\' se interpreta como escape
ruta_normal = "C:\nueva\carpeta\archivo.txt"
print(ruta_normal)
# Salida: C:
#         ueva\carpeta\archivo.txt

# Con cadena cruda (prefijo 'r' o 'R'), '\' es literal
ruta_cruda = r""C:\nueva\carpeta\archivo.txt""
print(ruta_cruda)
# Salida: "C:\nueva\carpeta\archivo.txt"
```

Cadenas crudas en Python (Raw strings)

Una **secuencia de escape** es una **combinación especial de caracteres** que **comienza con una barra invertida (\)** y que Python (y muchos otros lenguajes) interpreta de una forma distinta a como está escrita.

Su función es **representar caracteres que no pueden escribirse directamente en una cadena**, como saltos de línea, tabulaciones, comillas dentro de comillas, o incluso caracteres invisibles.

Secuencia	Significado	Ejemplo	Salida visible
\n	Salto de línea	"Hola\nMundo"	HolaMundo
\t	Tabulación (espacios grandes)	"Hola\tMundo"	Hola Mundo
\\	Barra invertida literal (\)	"C:\\Users\\Alumno"	C:\Users\Alumno
\'	Comilla simple literal	'It\'s a test'	It's a test
\"	Comilla doble literal	"Dijo: \"Hola\""	Dijo: "Hola"
\r	Retorno de carro (cursor al inicio de la línea)	"Hola\rMundo"	Mundo
\b	Retroceso (borra el carácter anterior)	"Hola\bMundo"	HolMundo
\f	Salto de página (menos usado)	"Hola\fMundo"	Hola␌Mundo
\uXXXX	Carácter Unicode (hexadecimal)	"\u00E1"	á
\UXXXXXXXX	Carácter Unicode extendido	"\U0001F600"	😀 (emoji)

Cadenas crudas (Raw strings)

Aunque las cadenas crudas son muy útiles, hay algunos detalles importantes a tener en cuenta. El principal es el siguiente:

No puedes terminar una cadena cruda con una barra invertida suelta:

Si la cadena termina con `\`, Python no sabrá si forma parte de la cadena o si está escapando la comilla final.

```
# Error de sintaxis
ruta = r"C:\nueva\carpeta\"
```

Solución:

- Añadir otra barra invertida para dejarlo claro:

```
ruta = r"C:\nueva\carpeta\\"
```

- O simplemente usar una cadena normal y escapar (usar la secuencia de escape) todas las barras:

```
ruta = "C:\\nueva\\carpeta\\"
```

Métodos de cadena más utilizados

Todos los métodos devuelven **nuevas cadenas**; la original no cambia.

Posición de caracteres

```
texto = "Hola qué tal"
texto.index("q")  # 5
texto.index("qué")  # 5
texto.index("a", 4)  # 10 (busca desde pos. 4)
```

Conversión de mayúsculas/minúsculas

```
s = "Programación"
s.lower()  # 'programación'
s.upper()  # 'PROGRAMACIÓN'
s.title()  # 'Programación'
s.capitalize()  # 'Programación'
```

Quitar espacios/caracteres

```
" hola ".strip()  # 'hola'
" hola ".lstrip()  # 'hola '
" hola ".rstrip()  # ' hola'
"---hola---".strip("-")  # 'hola'
```

Buscar y contar

```
s = "banana"
s.find("na")  # 2 (primer índice o -1 si no está)
s.count("na")  # 2
s.startswith("ba")  # True
s.endswith("na")  # True
```

Reemplazo

```
"hola mundo".replace("mundo", "clase") # 'hola clase'
"aaaa".replace("a", "b", 2) # 'bbaa'
```

Listas (list)



¿Qué son las listas?

En programación muchas veces necesitamos **guardar varios datos relacionados**: por ejemplo, las notas de una clase, los nombres de varios usuarios o los productos de una tienda online.

Una **lista** (list) es una colección **ordenada** y **mutable** de elementos.

Características principales:

- **Mantienen el orden** en el que añadimos los elementos
- **Podemos modificarlas**: añadir, quitar, ordenar, invertir, etc.
- Pueden contener diferentes **tipos** (números, cadenas, incluso otras listas)

¿Cuándo usar listas?

- Cuando necesitamos **mantener el orden de los datos**
- Cuando queremos **modificar la colección** (añadir, quitar, ordenar)
- Cuando necesitamos **acceder a elementos por posición** (índices)
- Para almacenar **colecciones dinámicas**, cuyo tamaño puede cambiar durante la ejecución del programa

Si los datos no deben cambiar, es mejor usar una tupla (tuple), que veremos más adelante.

Cómo declarar listas

Listas literales con corchetes: []

```
vacia = []
numeros = [1, 2, 3]
mixta = [1, "dos", 3.0, True]
anidada = [[1, 2], [3, 4]]
```

Mediante un constructor

La función **list()** convierte cualquier objeto iterable en una lista.

```
desde_iterable = list("hola")  # ['h','o','l','a']
desde_rango = list(range(5))  # [0,1,2,3,4]
```

Un **iterable** es **cualquier objeto en Python que se puede recorrer elemento por elemento**, como si fuera una fila de cosas ordenadas.

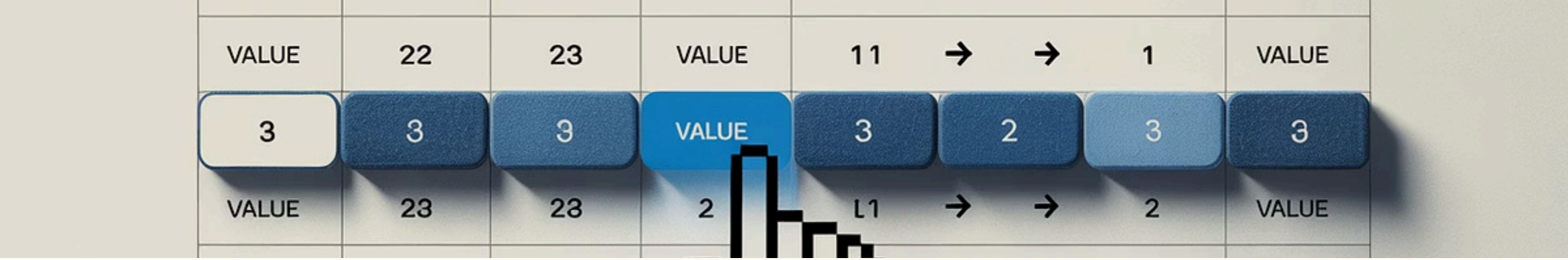
Ejemplos prácticos

```
# Lista de alumnos
alumnos = ["Ana", "Luis", "María", "Carlos"]

# Lista de notas
notas = [8.5, 7.9, 9.2, 6.8]

# Lista mixta con datos de un alumno
alumno_datos = ["Ana García", 20, 8.5, True]

# Lista vacía para añadir elementos después
productos = []
```



Acceso a elementos de una lista: Índices

Cada elemento tiene una **posición** (índice) que comienza en 0.

Elemento	1	2	3	4
Índice	0	1	2	3

```
colores = ["rojo", "verde", "azul", "amarillo"]
print(colores[0])  # rojo
print(colores[2])  # azul
```

Índices negativos

Los índices negativos permiten contar desde el final:

- -1 → último elemento
- -2 → penúltimo, etc.

```
print(colores[-1])  # amarillo
print(colores[-2])  # azul
```

Acceso a elementos de una lista: slicing

El **slicing** permite extraer **sublistas** con la sintaxis:

```
lista[inicio:fin:paso]
```

- **inicio** → posición desde donde empieza (incluida)
- **fin** → posición donde termina (**excluida**)
- **paso** → intervalo (opcional)

```
numeros = [10, 20, 30, 40, 50]
print(numeros[1:4]) # [20, 30, 40]
print(numeros[:3]) # [10, 20, 30]
print(numeros[2:]) # [30, 40, 50]
print(numeros[::-1]) # [50, 40, 30, 20, 10] (lista invertida)

numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(numeros[::2]) # [0, 2, 4, 6, 8] (desde el inicio hasta el final, saltando de 2 en 2)
print(numeros[1:8:2]) # [1, 3, 5, 7] (desde el índice 1 al 7 (8 no incluido), saltando de 2 en 2)
print(numeros[8:2:-1]) # [8, 7, 6, 5, 4, 3] (desde índice 8 hasta índice 3, retrocediendo)
```

Operaciones básicas con listas

Operación	Ejemplo	Resultado
Concatenación	[1,2] + [3,4]	[1,2,3,4]
Repetición	["ha"] * 3	['ha','ha','ha']
Longitud	len([1,2,3])	3
Pertenencia	3 in [1,2,3]	True
Máximo y mínimo	max([1,2,3])	3
Suma	sum([1,2,3])	6

Ejemplos prácticos

```
lista1 = [1, 2, 3]
lista2 = [4, 5]
combinada = lista1 + lista2 # [1, 2, 3, 4, 5]

repetida = [0] * 5 # [0, 0, 0, 0, 0]

if "manzana" in ["pera", "manzana", "naranja"]:
 print("La manzana está en la lista") # Se imprimirá
```

Métodos de listas: Añadir elementos

Método	Uso
append(x)	Agrega un elemento al final
extend(iterable)	Agrega varios elementos al final
insert(i, x)	Inserta un elemento en la posición i

```
numeros = [1, 2, 3]
numeros.append(4)     # [1, 2, 3, 4]
numeros.extend([5, 6]) # [1, 2, 3, 4, 5, 6]
numeros.insert(1, 10)  # [1, 10, 2, 3, 4, 5, 6]
```

Diferencias clave

- **append**: añade el objeto como un solo elemento
- **extend**: añade cada elemento del iterable por separado
- **insert**: permite elegir la posición donde insertar

Métodos de listas: Eliminar elementos

Método	Uso
remove(x)	Elimina la primera aparición de x
pop(i)	Elimina y devuelve el elemento en i. Si no se indica i, elimina el último
clear()	Borra todos los elementos
del lista[i]	Borra por índice sin devolver nada

```
numeros = [10, 20, 30, 40]
numeros.remove(20)  # [10, 30, 40]

ultimo = numeros.pop()  # ultimo = 40, numeros = [10, 30]

del numeros[0]  # numeros = [30]

numeros.clear()  # []
```

¿Cuál usar?

- **remove**: cuando conoces el valor pero no su posición
- **pop**: cuando quieres usar el elemento eliminado
- **del**: cuando conoces el índice y no necesitas el valor
- **clear**: cuando quieres vaciar toda la lista

Métodos de listas: Buscar y ordenar

Buscar elementos

Método	Uso
index(x)	Devuelve el índice de la primera aparición de x
count(x)	Cuenta cuántas veces aparece x

```
frutas = ["manzana", "pera", "pera", "naranja"]
print(frutas.index("pera"))  # 1
print(frutas.count("pera"))  # 2
```

Ordenar y revertir

Método	Uso
sort()	Ordena la lista in situ (no crea otra lista)
sorted(lista)	Devuelve una nueva lista ordenada
reverse()	Invierte el orden de la lista

```
numeros = [3, 1, 2]
numeros.sort() # [1, 2, 3]
numeros.sort(reverse=True) # [3, 2, 1]

original = [5, 3, 8]
ordenada = sorted(original) # original = [5, 3, 8], ordenada = [3, 5, 8]
```

Listas anidadas

Una **lista anidada** es una lista que contiene otras listas como elementos, permitiendo organizar los datos en varias dimensiones.

Ejemplos de uso en la vida real

- Una **tabla de notas**, donde cada fila es un alumno y cada columna es una asignatura
- Un **tablero de ajedrez**, representado por una cuadrícula 8×8
- Una **matriz** para cálculos matemáticos
- Datos organizados por categorías, como una lista de listas de productos

Representación de una tabla

```
datos = [
    ["Ana", 20, 8.5],
    ["Luis", 21, 7.9],
    ["María", 19, 9.2]
]
```

Aquí, **datos** es una lista que contiene **tres sublistas** (cada sublista representa una fila).

Acceso a elementos

Para acceder a elementos, usamos **índices encadenados**:

```
matriz = [
    [1, 2, 3],  # Fila 0
    [4, 5, 6],  # Fila 1
    [7, 8, 9]   # Fila 2
]
print(matriz[0][1])  # 2
print(matriz[2][2])  # 9
```

Como las listas son **mutables**, podemos cambiar valores directamente:

```
matriz[1][2] = 99
print(matriz) # [[1, 2, 3], [4, 5, 99], [7, 8, 9]]
```



Diccionarios (dict)

¿Qué son los diccionarios?

En muchos problemas necesitamos **buscar datos por un nombre** (o clave) en lugar de por una posición. Piensa en:

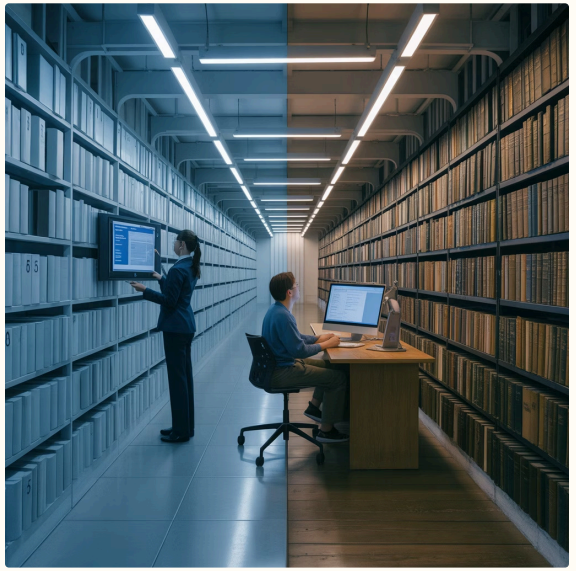
- un **contacto por su nombre**
- el **precio de un producto por su código**
- los **créditos** de un módulo por su **abreviatura**
- las **notas** de un alumno por su **DNI**

Un **diccionario** es una **colección mutable** (que puede cambiar) de pares **clave → valor**, donde:

- cada **clave** debe ser **hashable** (inmutable)
- los **valores** pueden ser de cualquier tipo (incluidos otros diccionarios)

¿Cuándo usar diccionarios?

- Cuando necesitas **acceso directo por nombre** o identificador
- Para **contar**, **agrupar** o **indexar** datos
- Para **modelar entidades** (usuario, producto, alumno) con atributos



Crear diccionarios

Puedes crear diccionarios de varias formas, todas equivalentes:

Literal con llaves {}

Para definir un diccionario usaremos las llaves **{}**. Separamos los elementos con comas **","** y para separar la clave del valor utilizaremos dos puntos **":"**.

```
vacio = {}
alumno = {"nombre": "Ana", "edad": 20, "nota": 9.2}
```

Constructor dict()

```
d1 = dict(nombre="Ana", edad=20)  # claves como identificadores

d2 = dict([("nombre", "Ana"), ("edad", 20)])  # a partir de pares (clave, valor)
```

A partir de parejas

```
pares = [("ES", "España"), ("FR", "Francia")]
paises = dict(pares)  # {'ES': 'España', 'FR': 'Francia'}
```

Ejemplo práctico

```
# Diccionario de contactos
contactos = {
 "Ana": "612345678",
 "Luis": "698765432",
 "María": "634567890"
}

# Diccionario de precios de productos
precios = {
 "manzana": 0.85,
 "plátano": 1.20,
 "naranja": 0.90
}
```



Acceso, inserción y actualización

El acceso por clave es el motivo principal para usar dict. Hay dos estilos: **directo** (lanza error si no existe) o **seguro** (con valor por defecto).

Acceso a valores

```
alumno = {"nombre": "Ana", "nota": 9.2}

# Acceso directo
alumno["nota"]  # 9.2
alumno["edad"]  # Generará un Error dado que la clave edad no existe

# Acceso seguro
alumno.get("edad")  # None
alumno.get("edad", 0)  # 0 (por defecto)
```

Insertar / actualizar

```
alumno = {"nombre": "Ana", "nota": 9.2}

# Insertar nueva clave-valor
alumno["grupo"] = "A" # inserta la clave "grupo" y el valor "A"

# Actualizar valor existente
alumno["nota"] = 9.5 # actualiza la clave "nota" con el nuevo valor

# Actualizar varios campos
alumno.update({"curso": "SMR", "nota": 9.7})
```

Métodos frecuentes de diccionarios

Estas funciones devuelven **vistas dinámicas** del diccionario: si el diccionario cambia, **la vista se actualiza**.

Obtener vistas

```
d = {"a": 1, "b": 2}
ks = d.keys()    # dict_keys(['a','b'])
vs = d.values()  # dict_values([1,2])
it = d.items()   # dict_items([('a',1), ('b',2)])

# Si añadimos un elemento, las vistas se actualizan
d["c"] = 3
list(ks)  # ['a','b','c'] ← reflejan el cambio
```

Eliminar elementos

```
d = {"a": 1, "b": 2, "c": 3}

# Eliminar y obtener el valor
v = d.pop("b")  # elimina 'b' y devuelve 2 en la variable v
v = d.pop("x", 0)  # devuelve 0 si no existe, sin error

# Eliminar sin devolver valor
del d["a"]  # borra por clave, sin devolver valor

# Vaciar diccionario
d.clear()  # deja el dict vacío
```

Una vista dinámica es como una "ventana" que te permite ver las claves o valores de un diccionario en **tiempo real**. Si el diccionario cambia, esta ventana muestra los cambios automáticamente sin tener que crear una copia nueva.

Diccionarios anidados

Igual que hemos visto con las listas, los diccionarios nos permiten incluir otros diccionarios en su interior, de una manera anidada. Son muy útiles para representar **objetos con subobjetos** (por ejemplo, JSON).

```
usuario = {
    "nombre": "Ana",
    "perfil": {
        "email": "ana@ejemplo.com",
        "edad": 20,
    },
    "cursos": ["BD", "Sistemas",
"Entornos"]
}
```

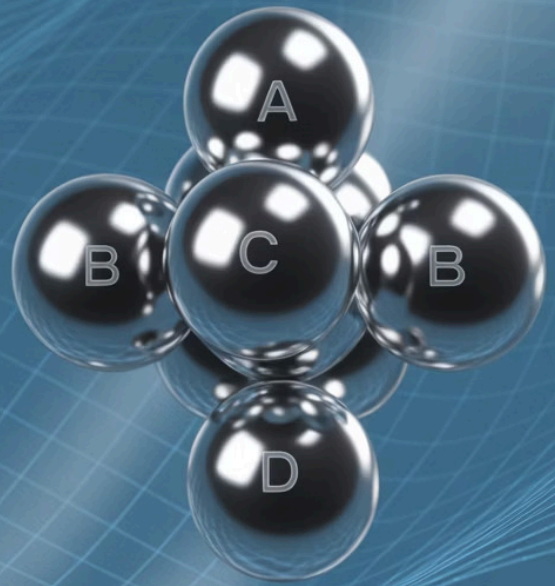
Acceso a elementos anidados

```
# Acceso directo
usuario["perfil"]["email"]  #
'ana@ejemplo.com'

# Acceso seguro con get()
email = usuario.get("perfil",
{}).get("email", "desconocido")
```

El acceso seguro es útil cuando no estamos seguros de si existe la clave o subclave.





Tuplas (tuple)

¿Qué son las tuplas?

En programación, muchas veces necesitamos **agrupar varios datos en una sola unidad lógica**, como si fueran un solo elemento.

Por ejemplo:

- Las **coordenadas** de un punto (x, y)
- Los **datos personales** de un usuario: nombre, edad y teléfono
- El **resultado** de una función que devuelve varios valores

Una **tupla (tuple)** es una colección ordenada e inmutable de elementos que puede contener datos de distintos tipos.

Características

- **Colección ordenada:** mantiene el orden en que se añaden los elementos
- **Indexable:** se accede a sus elementos por posición, empezando en 0
- **Inmutable:** no se puede cambiar una vez creada
- **Heterogénea:** puede contener datos de cualquier tipo
- **Anidable:** puede contener tuplas dentro de otras tuplas

Idea clave: Una tupla es como una lista, pero **inmutable**, es decir, después de crearla, **no se puede modificar**, ni añadir, ni eliminar elementos.

Diferencias entre tuplas y listas

CARACTERÍSTICA	LISTA (LIST)	TUPLA (TUPLE)
MUTABILIDAD	Sí, se puede modificar	No, es inmutable
SINTAXIS	[elementos]	(elementos)
USO TÍPICO	Datos que cambian con el tiempo	Datos fijos, constantes
TAMAÑO	Dinámico	Fijo después de crearla
VELOCIDAD	Más lenta (por flexibilidad)	Más rápida (por ser inmutable)
USO COMO CLAVE EN DICT O ELEMENTO DE SET	No, porque es mutable	Sí, si sus elementos son inmutables

Conclusión práctica

- Si los datos deben **cambiar**, usa una **lista**
- Si los datos son **constantes** y no deben modificarse, usa una tupla

Cómo declarar tuplas

Literal con paréntesis

La forma más común es poner los elementos separados por comas dentro de paréntesis.

```
vacia = ()  # Tupla vacía
coordenadas = (10, 20)  # Tupla con dos elementos
datos = ("Ana", 20, 8.5)  # Elementos de distintos tipos
un_elem = (100,)  # Solo con un elemento, la "coma" es obligatoria
```

Sin paréntesis (solo con comas)

Python permite crear tuplas **solo con comas**, sin necesidad de paréntesis.

```
tupla = 1, 2, 3
print(tupla)  # (1, 2, 3)
```

Esto se usa mucho en asignaciones múltiples y en el retorno de funciones.

Constructor tuple()

Puedes crear una tupla a partir de cualquier **iterable** (lista, cadena, rango, etc.).

```
letras = tuple("Python") # ('P', 'y', 't', 'h', 'o', 'n')
numeros = tuple([1, 2, 3]) # (1, 2, 3)
```



Acceso a elementos en una tupla

Acceso por índice

Las tuplas son **indexables**, igual que las listas. El primer elemento tiene índice 0.

```
colores = ("rojo", "verde", "azul")
print(colores[0])  # rojo
print(colores[1])  # verde
print(colores[-1])  # azul (índice
negativo: empieza desde el final)
```

Subtuplas con slicing

Puedes obtener una **subtupla** usando la misma sintaxis de slicing que con listas:

```
tupla[inicio:fin:paso]


numeros = (0, 1, 2, 3, 4, 5)
print(numeros[1:4])  # (1, 2, 3)
print(numeros[:3])   # (0, 1, 2)
print(numeros[::2])  # (0, 2, 4)
```

Recuerda: el slicing **no modifica la tupla original**, devuelve una **nueva tupla**.

Inmutabilidad de las tuplas

Una vez creada, la tupla **no puede cambiar**:

- No se pueden añadir elementos (append no existe)
- No se pueden eliminar elementos
- No se pueden modificar elementos por índice

```
coordenadas = (10, 20)
coordenadas[0] = 30  # TypeError: 'tuple' object does not support item assignment
```

Mutabilidad dentro de la inmutabilidad

Aunque la tupla es inmutable, **puede contener objetos mutables** en su interior. Lo que no se puede cambiar es **qué elementos contiene la tupla**, pero **sí se pueden modificar esos objetos internos**.

```
tupla = (1, [2, 3])
tupla[1].append(4)
print(tupla)  # (1, [2, 3, 4])
```

Como observamos, **no se ha modificado la estructura de la tupla**, sigue teniendo dos elementos, el 2º sigue siendo una lista, pero la lista sí es mutable, por lo que podemos modificarla.

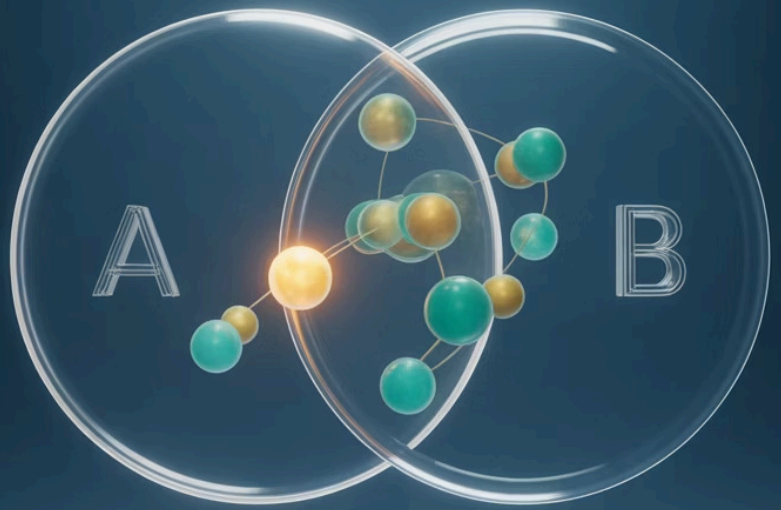
Operaciones con tuplas

Operación	Ejemplo	Resultado
Concatenación	(1, 2) + (3, 4)	(1, 2, 3, 4)
Repetición	("a",) * 3	('a', 'a', 'a')
Longitud	len((1, 2, 3))	3
Pertenencia	2 in (1, 2, 3)	True
Máximo y mínimo	max((1, 2, 3))	3
Suma	sum((1, 2, 3))	6

Métodos de las tuplas

Las tuplas tienen **solo dos métodos**, debido a su inmutabilidad:

Método	Descripción	Ejemplo
.count(x)	Devuelve cuántas veces aparece x	(1, 2, 2, 3).count(2) → 2
.index(x)	Devuelve la primera posición de x	(1, 2, 3).index(2) → 1



Conjuntos (set y frozenset)

¿Qué son los conjuntos?

En matemáticas, un conjunto es una colección de elementos **sin duplicados**. En Python, un **set** funciona igual: nos permite almacenar valores únicos y realizar **operaciones de teoría de conjuntos** como unión, intersección y diferencia.

Un **set** es una colección **mutable**, **no ordenada** y **sin elementos duplicados**. Es ideal para pruebas de pertenencia rápidas y para eliminar duplicados.

frozenset es su versión **inmutable** (hashable o que no se puede modificar).

¿Cuándo son útiles?

- Eliminar duplicados de una lista de datos
- Hacer pruebas rápidas de **pertenencia** ("¿Está este elemento aquí?")
- Comparar grupos de elementos y trabajar con datos únicos
- Realizar operaciones matemáticas de conjuntos

Importante: No ordenada significa que **no puedes indexar ni hacer slicing**. Para mostrar en orden usa **sorted(s)**, que devuelve una **lista** ordenada.

Cómo declarar conjuntos

Con llaves {}

Como los diccionarios (pero **{} vacío es un dict**, no un set), pero sin el par clave-valor:

```
# Set con elementos
s = {1, 2, 3}

# ¡Ojo! {} es un dict vacío, NO un set
```

Con el constructor set()

```
# A partir de un iterable
s2 = set([1, 1, 2, 3])  # {1, 2, 3}
s3 = set("hola")  # {'h', 'o', 'l', 'a'}

# Set vacío (no se puede usar {})
s_vacio = set()
```

Frozenset (inmutable)

```
fs = frozenset([1, 2, 2, 3])  #
frozenset({1, 2, 3})
```

Los frozenset son útiles cuando necesitas un conjunto inmutable, por ejemplo, para usarlo como clave en un diccionario.

Ejemplo práctico: eliminar duplicados

```
numeros = [1, 2, 2, 3, 3, 3, 4]
numeros_unicos = list(set(numeros)) # [1, 2, 3, 4]
```

Operaciones y métodos de conjuntos

Operaciones de teoría de conjuntos

OPERACIÓN	OPERADOR	EJEMPLO
UNIÓN	a | b	{1,2} | {2,3} -> {1,2,3}
INTERSECCIÓN	a & b	{1,2} & {2,3} -> {2}
DIFERENCIA	a - b	{1,2,3} - {2} -> {1,3}
DIF. SIMÉTRICA	a ^ b	{1,2} ^ {2,3} -> {1,3}

Métodos esenciales de set

```
s = {1, 2, 3}
# Añadir / ampliar
s.add(4)  # {1,2,3,4}
s.update([4,5], {6,7})  # admite varios iterables

# Quitar
s.remove(2)  # KeyError si no está
s.discard(99)  # NO da error si no está
x = s.pop()  # quita y devuelve un elemento "cualquiera"
s.clear()  # deja el set vacío

# Consultas
s.issubset(t)
s.issuperset(t)
s.isdisjoint(t)
```

Ejemplo práctico con sets

```
smr = {"Ana", "Luis", "Zoe"}
daw = {"Zoe", "Mario"}

alumnos = smr | daw # unión
solo_smr = smr - daw # diferencia
en_ambos = smr & daw # intersección
exclusivos = smr ^ daw # dif. simétrica
```

¡Gracias por tu atención!

Recuerda: las **cadenas**, **listas**, **diccionarios**, **tuplas** y **conjuntos** son herramientas fundamentales que potenciarán tus proyectos.

¿Alguna pregunta?

